

Yield Estimation using Deep Learning for Precision Agriculture

Youssef Osman*, Reed Dennis*, Khalid Elgazzar

Ontario Tech University

{youssef.osman, reed.dennis, khaled.elgazzar}@ontariotechu.ca

Abstract—We perform fruit counting on video footage by following a two-stage pipeline that consists of detecting the fruits, then tracking them frame-by-frame. Detection is done through the use of You Only Look Once model (YOLO). Bounding boxes are extracted from detection and Non Max Suppression (NMS) is performed to get final detections. The boxes are then input into the tracking pipeline. For tracking, we apply a custom-developed DeepSORT algorithm to work with fruits. Using the box coordinates, every detected object is cropped out of the original image, and a separate feature extraction using a convolutional neural network (CNN) called ResNet is performed on that image crop to get the feature map. New detections are associated with old detections by comparing their features as a distance metric, where two objects with minimal distance are associated together. Input objects with no association are treated as new objects to be tracked. By keeping track of the fruits throughout the video frames, we ensure that we're counting them appropriately when they are first detected. We demonstrate the approach on videos from an apple orchard to test the performance of the proposed pipeline in natural light. Experimental results show high accuracy of fruit counting on real-time video feeds. The new approach can be efficiently applied on any type of fruit and vegetables with no changes in the algorithms.

I. INTRODUCTION

Precision agriculture, where robotics and AI technologies are used in various aspects of crop field management, is an ever-growing domain. Technologies such as Global-Positioning-System (GPS), wireless sensing, image processing and AI can be used for surveying and analysis of various crop fields. For example, Corwin [1] uses soil sensors to measure the electrical conductivity of the soil, and explains that this property highly influences crop productivity. Tian [2] presents a sensor capable of identifying weed zones. Obtaining crop information and automating processes such as fertilizing and treatment dispensing is proven to have positive impacts in improving a farmer's efficiency and profits [3]. For example, if farmers know information such as where certain areas are affected by weeds and diseases, they can quickly isolate the areas before hurting the rest of the crops, and prepare for treatment. Another example, if farmers know their yield when the crop is still maturing, they can accurately calculate their harvest and production costs, and their profits, and set up their equipment in advance. All this information directly benefits a farmer's profitability and efficiency. In this paper, we specifically target yield estimation, and the use of AI and deep learning to perform fruit counting.

Fruit counting, which can also be referred to as crop yield estimation, is the process of producing an estimated count of crops. Knowing the yield helps farmers make informed choices in terms of how to allocate resources for harvesting, preparing storage according to crop sizes, and better plan for harvest routes and packaging. Given the tedious nature of having people go out into the field and count, and a desire for fast and accurate estimations, automating this process would be greatly beneficial. In many works, computer vision can be used to provide counts of crops from image data. In the era of advanced robotics a device such as an unmanned aerial vehicle (UAV) can be used to a great extent in acquiring footage or images of the crops [4], given that they're normally equipped with RGB cameras, and can navigate between plants and trees while recording. On the footage, a computer vision technique that detects the crops based on features such as texture or colours is utilized [5]. However, there are a number of limitations with such techniques. Notably, they're very fruit-specific, as in what works for one fruit may not work for another. They also struggle with natural feature changes, like illumination or occlusion. Overlapping fruits being miscounted or ignored also pose a challenge.

In order to address such issues, deep learning is used for image analysis. Not only are deep learning models quite accurate, but they're also highly versatile [6]. The same deep learning technique can classify different types of fruit using the same model, given the appropriate dataset is used for training. The dataset can also include the fruits in different lighting conditions or partially occluded, such features can be learned and detected using the model as we will show in our work. Deep learning has already been successfully used for fruit detection and counting [7] [8] [9], however, the function was performed on still images. Footage of fruit taken in video frames is substantially more practical, as it's easier to set up a robot or vehicles traversing through the fields to capture recordings. The videos also provide multiple angles of the fruit, allowing occluded or shadowed fruits in one frame to be clear in other frames. Performing counting on videos is not possible using just detection, as there is no way of telling whether an apple detected in the first frame is the same apple detected in the following frame. Thus, if we perform counting using detection alone, we'd be counting the same apples multiple times. To address this challenge, we design a pipeline where we insert our detection into a tracking

mechanism, and perform counting on the tracked fruits. By using tracking, we associate between detections in different frames. This allows us to identify individual apples, keep track of them across the video frames, and count every tracked apple only once. In this paper, we leverage deep learning by using a robust detection model, and develop a lightweight correction mechanism to handle incorrect apple detections in trees from behind rows. We then integrate our detections with an object tracking pipeline that we adapt to work with fruits so that the counting task can be done on video footage. We successfully run our pipeline on footage of a row of apple trees taken from an apple orchard.

The remainder of the paper is organized as follows. Section 2 discusses related works in fruit detection and counting. Section 3 describes our two-stage pipeline. We explain how we prepare our dataset and present our results in Section 4. Lastly, Section 5 provides concluding remarks and outlines future work.

II. RELATED WORKS

Crop yield estimation relies on using image processing algorithms detect the crop from imagery and count the number of detections to give the estimate. In earlier works, detections are made by distinguishing between the crop's textures and background textures. Wang [10] uses visual cues to segment apples from images, specifically the hue, saturation and intensity to detect both red and green apples under specific lighting. Pothen [11] presents a keypoint detection algorithm that detects potential fruit regions based on intensity profile then uses high-dimensional features in those detected regions to classify them as fruit or not fruit. Roy [12] estimates apple count from images as well as their diameters by segmenting apple clusters in an input image using a nonlinear optimization method. Then feed the segmented images into a Structure from Motion (SfM) pipeline to reconstruct them up to scale. The aforementioned techniques do not produce state-of-the-art results, are very specific to the selected crop (e.g., apples), and cannot be easily translated into yield estimation for other types of crop without a drastic change in the algorithm. Additionally, these algorithms can perform poorly outside of controlled environments, such as light intensity. In our work, the proposed models can be used with any fruit, given the appropriate dataset is provided to train the model in different circumstances, such as poor lighting conditions and occlusion behind another fruit or leaves.

Deep learning is on the rise in computer vision, with numerous state-of-the-art object detection and image segmentation algorithms utilizing deep learning models showing promising accuracy. Furthermore, one of the main advantages of deep learning is generalization, if the model works on one type of object, then it's quite likely to work on other objects. It only needs to be retrained with a suitable dataset relevant to the domain of application. Rahnemoonfar [9] prove this by performing fruit detection on 7 different fruits using the Faster R-CNN object detector which is trained on the respective fruit. Bargouti [13] use Multi-Layered Perceptrons

and Convolutional Neural Networks (CNNs) to perform image segmentation to detect and count fruits from images. Sa [7] performs tomato counting using a CNN model that's a modification of Inception-ResNet [14]. Notably, the data they used for training is simulated by generating synthetic images of tomatoes, producing favorable test accuracy on real images of tomatoes. Chen [15] present a deep learning pipeline for counting apples and oranges. An image of the crop is input into a blob detection neural network that is a fully convolutional network [16] which outputs a segmented image to distinguish fruit pixels from non-fruit pixels, noting that there could be a cluster of fruits that's referred to as a blob. They then use a count neural network, which is another CNN that takes bounding boxes around each detection, and estimates the count of fruit in that box. These works produce great accuracy with good performance and potential for generalization, showing that deep learning is a viable methodology for yield estimation. However, they all work on images, not videos, of crops. This is an important issue to discuss because, as footage of the crop is typically taken in video form. In our work, we use the detection model within a tracking pipeline, which allows the system to keep track of detections across video frames and count new detections only once.

Liu [17] addresses counting through videos by using a fully convolutional neural network (FCN) for object detection, and optical flow using Kanade-Lucas-Tomasi (KLT) tracker. However, their tests on apples were run under controlled illumination. Moreover, KLT-based optical flow tracking has shortcomings: it relies on the brightness of different regions in the frames remaining consistent, and that the object motion in that region has to be consistent. Any change of lighting or sudden change of motion will cause the tracker to suffer. Their work also included the use of SfM to avoid counting apples in different tree rows, which works well but the algorithm itself is very computational intensive. In our approach, we use a single-shot-detector (SSD) which is considered the state of the art solution to object detection problems. Additionally, we developed a custom tracking pipeline that leverages deep learning and intersection over union (IoU) matching in order to properly track objects through sudden motions, and changes in appearances. We also design a light-weight solution to resolve apples in back rows being unintentionally detected. Our dataset and tests are run on apple trees in natural illumination during daytime.

III. METHODOLOGY

In this section, we discuss the proposed pipeline used to count fruits. Using a detection model is insufficient on a video containing the fruits to perform counting, as there is no way of knowing whether the fruit detected in the first frame is the same as the fruit detected in the second frame and should not be counted again. As such, we utilize tracking, where fruits detected in each frame are associated with each subsequent frame based on similarity metric, allowing their proper identifications. Our pipeline is divided into two main parts: *Detection* and *Tracking*.

A. Detection

Object detection is the process of identifying objects in an image, classifying them, and returning their pixel-wise location in the image. This process is accomplished by drawing bounding boxes around desired objects in an image, these bounding boxes use coordinates that denote where they lie on the image. In our case, the object detector must be able to acquire the location of apples. The model we selected for detection is YOLOv3 [18].

1) *Model Selection:* Lately, convolutional neural network (CNN) based object detectors have been able to achieve state-of-the-art performance on various benchmark datasets. As a result of the success of CNNs we decided to use CNNs over a hand-crafted feature extractor. The basic structure of a CNN object detector contains two parts, the backbone [19]–[24] which is responsible for extracting features from the image(s), and the head [18], [25]–[27] which is responsible to predict bounding box locations and class values. Recently, CNNs have been fit with intermediate blocks between the backbone and the head; commonly called the neck [28]–[33] which is responsible for collecting feature maps from multiple stages of feature extraction or feature refinement for feature layers. Commonly, the goal of the intermediate neck layer/blocks is to improve accuracy whilst minimizing the incurred overhead.

We chose to use YOLOv3 with spatial pyramid pooling (YOLOv3-SPP) [18] as the back-end object detector for our pipeline. Table I shows that YOLOv3-SPP has the best trade-off for accuracy and speed compared to other current state-of-the-art object detectors. Note that while the SpineNet [19] backbone using a Retina [26] prediction network as its head does provide very promising results, the results are somewhat skewed by the use of NVIDIA TensorRT [34] (TRT). With the use of TensorRT you could expect a network's inference time to decrease without affecting the accuracy. We do not use TensorRT in our work because it's still highly experimental. The framework is very hardware demanding as it only works on certain NVIDIA GPUs, and is only compatible with limited CUDA versions.

2) *Model Architecture:* You Only Look Once (YOLO) object detector is divided into two components: feature extractor (the backbone) and detector (the head). The feature extractor is responsible for getting a feature map of the image. YOLO uses a CNN model called DarkNet53 that uses 53 layers as shown in Fig. 1. The architecture is fairly straightforward with multiple convolutional layers to process the input, and residual layers, which are also known as skip layers, in-between. Convolutional layers are responsible for analyzing the image using filters with varying sizes. The filter, which is considered the weight matrix of the convolutional neural network, slides across the two dimensional input, multiplying with the input, then slides to the next window based on the stride parameter. The output of the layer can be considered a mathematical representation of the image that's used to classify the object within the image. Theoretically, the deeper the network is, the more accurate the classification will be. However, that's

not always the case, hence the introduction of residual layers [36] which are also used in the YOLO model. Where in traditional neural networks, each layer feeds directly into the next layer, residual networks allow for some layers to be skipped which notably helps in combating vanishing gradients—where during learning small differences eventually “vanish” in very deep networks because of how much they propagate. With skip connections, these small gradients can back-propagate through shorter paths, allowing for less changes. The outputs of convolutional layers are considered the feature maps, which are used in the next stage.

	Type	Filters	Size	Output
1x	Convolutional	32	3 × 3	256 × 256
	Convolutional	64	3 × 3 / 2	128 × 128
	Convolutional	32	1 × 1	
	Convolutional	64	3 × 3	128 × 128
2x	Residual			128 × 128
	Convolutional	128	3 × 3 / 2	64 × 64
	Convolutional	64	1 × 1	
	Convolutional	128	3 × 3	64 × 64
4x	Residual			64 × 64
	Convolutional	256	3 × 3 / 2	32 × 32
	Convolutional	128	1 × 1	
	Convolutional	256	3 × 3	32 × 32
8x	Residual			32 × 32
	Convolutional	512	3 × 3 / 2	16 × 16
	Convolutional	256	1 × 1	
	Convolutional	512	3 × 3	16 × 16
8x	Residual			16 × 16
	Convolutional	1024	3 × 3 / 2	8 × 8
	Convolutional	512	1 × 1	
	Convolutional	1024	3 × 3	8 × 8
4x	Residual			8 × 8
	Avgpool		Global	
	Connected		1000	
	Softmax			

Fig. 1: Darknet53 architecture that is typically used as YOLOv3's backbone as seen in [18]

Detectors in the YOLOv3 function are similar to anchor box object detectors such as Faster R-CNN [37]. YOLO leverages the output of each of the last 3 convolution stages. Three stages are selected to allow for multi-scale detection, where smaller objects are easier to detect in earlier stages, and bigger objects are easier to detect in later stages. Anchor boxes are assigned to each cell in the output matrix of the convolution stages. These anchor boxes have different aspect ratios and sizes. When learning, the anchor box with the best intersection with the ground truth bounding box is considered the correct prediction. Thus, when predicting each box, the information predicted will be: the location of the box, whether the box contains an object or not, and what class this object belongs to.

Non-max suppression (NMS) is applied after all boxes have been predicted. This is important because it's possible for the same object to be detected multiple times. The algorithm takes the box with the highest confidence, then compares it with all

Method	Backbone	AP	FPS (Batch Size of 1)	GPU
YOLOv3-512 [18]	Darknet-53 [18], [35]	42.4%*	48.7*	Telsa P100 (10 TFLOPS)
YOLOv3-608 [18]	Darknet-53 [18], [35]	43%*	43.1*	Telsa P100 (10 TFLOPS)
EfficientDet-D2 [25]	EfficientNet-B2 [20]	43.9%	56	Tesla V100 (15.7 TFLOPS)
EfficientDet-D3 [25]	EfficientNet-B3 [20]	47.2	34.4	Tesla V100 (15.7 TFLOPS)
RetinaNet [26] (w/TRT)	SpineNet-49S-640 [19]	39.9%	85.4	Tesla V100 (~30 TFLOPS)
RetinaNet [26] (w/TRT)	SpineNet-49-640 [19]	42.8%	65.3	Tesla V100 (~30 TFLOPS)
CenterNet-HG [27]	Hourglass-104 [23]	45.1%	7.8	Titan XP (12 TFLOPS)
CenterNet-DLA [27]	DLA-34 [24]	41.6%	28	Titan XP (12 TFLOPS)

TABLE I: Comparison between different SSD models

* Achieved via custom tests.

other potential boxes, if the intersection over Union (IoU) seen in Fig. 2 is above a certain threshold (0.6 in our case) then it is removed as it will be considered a redundant box that covers the same object. This step is performed on all predicted boxes. It's important to allow for some intersection to exist in apple detection, as having multiple apples overlapping occurs often in apple trees.

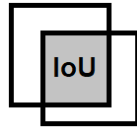


Fig. 2: IoU denotes how much two boxes overlap with each other, with 0.0 being no overlap, and 1.0 meaning they fully intersect

3) *Correcting Apple Detections*: In our implementation with apple data, we noticed that apples from other tree rows are being detected. This is not intended as apple orchards are divided into rows of trees, and we perform the counting task on one row at a time as our drone flies between the rows and provides the video feeds. To solve this problem, we developed a lightweight thresholding mechanism to correct the detections returned by YOLO. We tested the detection model standalone and analyzed the average bounding box sizes of the distant apples specifically, and set the bounding box height and width thresholds accordingly. Any bounding box that has a height and width below the threshold (30 pixels) is discarded before moving onto the next step.

For our work, we use YOLOv3 with default configurations to detect apples, where we obtain the x, y coordinates of the center of the box; the width and height of the box; the class it belongs to; and the class confidence score. All this information is then used for tracking. The use of default configurations allow us to improve the accuracy of detection directly through the use of transfer learning, where we can simply fine tune the original weights pretrained on the MSCOCO dataset which already contains apple data. The dataset and training will be discussed in Section 5A.

B. Tracking

We extract the bounding boxes from the detector, and perform a tracking function in order to keep track of the objects across the frames. This type of tracking function is considered a multi-object object tracking, or Multi-Hypothesis-Tracking (MHT) [38], where there are numerous objects in the frames that are tracked separately. The algorithm we use for MHT is a modified version of DeepSORT [39].

DeepSORT is an extension of the Simple Online Realtime Tracking (SORT) [40] algorithm. It relies on creating "tracks" which represent the objects to be tracked, and applying Kalman Filter to get states of objects. It then associates between the same objects in different frames by using distance metrics, one based on the motion of the object, and the other based on the appearance of the object. The model gives favorable performance only using the appearance as a metric [39], so for our work, we only use the features produced by the CNN which represent the appearance of the object.

Following the original SORT, DeepSORT performs Kalman Filtering, and defines the tracking scenario on an 8-dimensional state space $(x, y, a, h, \dot{x}, \dot{y}, \ddot{x}, \ddot{y})$ which respectively denote the center x and y coordinates of the bounding box, the aspect ratio and height, and their velocities. A linear Kalman Filter following a constant velocity model is used, which is standard for object tracking [41]. The bounding box coordinates (x, y, a, h) which are received from YOLO are used without modification. DeepSORT then uses deep learning and CNN features to create associations between new detections and the current tracked objects by using a minimum cost algorithm to obtain the objects with the minimum distance with their associated tracks.

1) *Track Initialization*: After we obtain the first bounding boxes from YOLO, the first step in the DeepSORT pipeline is initializing the track object for the detections. The track is given a number of attributes including:

- Track ID as an identifier
- Age which denotes how many frames this track existed in, initialized as 1

- Number of hits which is incremented every time this track is successfully associated, initialized as 1
- Feature matrix which is the output of the CNN and represents the object's appearance
- Track state which represents the current state of the track so it can be confirmed or deleted, initialized as tentative
- Mean and covariance matrices which are used for the Kalman Filter that are updated with every prediction

2) *Convolutional Neural Network*: The key point of DeepSORT is the use of deep convolutional features. When inputting an image into a CNN, the features of the images are analyzed within every layer, then once all the convolutional layers are finished, an activation function such as softmax is applied on the output vector to classify the object. Intuitively, this means that the vector output after convolutional layers describes the appearance of whatever is in the image. This was leveraged in YOLO in order to detect objects, and it's also used in DeepSORT's pipeline as the appearance metric for tracking. The use of appearance allows for better tracking in case of a random object motion and occlusion, which are both very common challenges in tracking applications.

The original model used is a wide residual network that's been trained on a person re-identification dataset [42] which is limited to people tracking (specifically, pedestrians) making it unsuitable for any fruit data. Thus, in this work we insert a different CNN model that shows state-of-the-art performance called ResNet18 [36]. We opted to select and implement ResNet, instead of building a CNN model from scratch for a number of reasons. Firstly, ResNet has been used in a wide variety of applications for years now, and its performance and drawbacks have been studied- its robustness is unmatched. Additionally, ResNet comes with a number of configurations, mainly different depths. This gives us the option to select a depth based on our needs. For our system, we implemented ResNet18, as it shows the fastest performance, and apple features are not complex enough to justify more layers. Most importantly, ResNet provides weights that are pretrained on the ImageNet classification dataset [43], a massive dataset with 1000 classes, including fruits such as apples for our application. This allows us to skip any complexity with training the feature extractor of DeepSORT, and allows the DeepSORT pipeline to be used for almost any fruit without needing to modify the tracking model or any further training.

Fig. 3 showcases the architecture of ResNet18. There are 5 convolutional layers, and a skip connection is in between every layer. After the final convolutional layer, there is an average pooling layer which leads to an output vector of size [1, 1, 512] that's input into a fully connected layer for classification. Since we do not perform classification and we need the appearance descriptor of the object, we take the output of the average pooling layer as our final output to be treated as the feature map of the object.

3) *Association and Counting*: Given a set of tracks, and their feature maps, and a set of detections and their feature maps, the model must know how to associate between them. This is an assignment problem to be solved using the Hungar-

Layer Name	Output Size	ResNet-18
conv1	$112 \times 112 \times 64$	$7 \times 7, 64, \text{stride } 2$
conv2_x	$56 \times 56 \times 64$	$3 \times 3 \text{ max pool, stride } 2$ $\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 2$
conv3_x	$28 \times 28 \times 128$	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 2$
conv4_x	$14 \times 14 \times 256$	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 2$
conv5_x	$7 \times 7 \times 512$	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 2$
average pool	$1 \times 1 \times 512$	$7 \times 7 \text{ average pool}$
fully connected	1000	$512 \times 1000 \text{ fully connections}$
softmax	1000	

Fig. 3: ResNet18 architecture that we use for DeepSORT's feature extraction, we discard the fully connect and softmax layers as seen in [36]

ian algorithm, which is a standard minimum cost algorithm. The problem shown in Eq. III-B3 is formulated as follows: given an array of tracks (T) of size t, and an array of detections (D) of size d, we compute a new cost matrix (C) where index [i, j] is the cosine similarity between the feature map of T(i) and the feature map of D(j). After the matrix is complete, we simply find the minimum cost for each track in T out of all the detections and that forms the association.

$$C(i, j) = \min(1 - [D(j)^T * T(i)])$$

There are a number of considerations though. First, a track is not immediately confirmed upon association. The track needs to have a number of hits greater than 2 to change its state to "confirmed". This helps to avoid brief false detections. Secondly, not all tracks will be associated. The distance (where 0.0 denotes an exact match, and 1.0 denotes completely different features) needs to meet a certain threshold to be considered as a match. Following the original implementation, the maximum distance is 0.15, meaning that for anything higher, the match will be discarded. If a track isn't associated, then it will remain saved for a number of frames until its age surpasses the preset maximum age, which we set as 30. In a 30 FPS video, if an object is not associated for 1 second, then it is also discarded as it is considered to have left the scene.

Before unmatched detections are initialized as new tracks, they undergo IoU matching first. Specifically, the IoU between two tracks bounding boxes is computed, and if the IoU is 0.3 or greater (meaning that there is some intersection), the tracks are associated with each other. This improves the tracking robustness because sometimes, an apple can be clear in one frame, obscured in another (thus having different features), then clear again in the following frame. The IoU tracking steps

Metrics	Pretrained Weights	Pretrained Weights with Correction	fine tuned Weights
Predicted/Ground Truth	523/342	299/342	313/342
L1 Loss	181	43	29
Accuracy	47.06%	87.43%	91.5%

TABLE II: Results of the proposed pipeline running on a video clip of apples. We show the predicted count versus the actual count and compute the L1 Loss and Accuracy. The accuracy was low with the pretrained weights due to a significant overcount. Our correction mechanism substantially improved the accuracy, however fine tuning the weights led to the best performance.

in and helps keeping track of that apple when an appearance association can briefly fail.

Another important parameter in DeepSORT includes a minimum confidence, which is the minimum accepted confidence from the detector (YOLO in this case). Anything lower than the minimum confidence causes the detection not to be inserted into the tracking pipeline. We selected a score of 0.3, as apples that were partially hidden by leaves or branches, or apples that were blurred due to camera motion were indeed detected, but with notably lower confidence, thus a lower threshold allowed for such apples to be taken into account. It was also observed that with thresholds lower than 0.3 started to include wrong detections, mostly leaves that were mistakenly detected as apples and have very low confidence scores. All thresholds were selected by referring to the non-modified implementation of DeepSORT and informal experimenting.

For counting, it's important to note that while every new track has its own ID, we cannot use the track ID as a reference for the count since some tracks remain tentative and become deleted if not properly associated. As such, the apples are counted once the track becomes confirmed.

IV. EXPERIMENTS AND RESULTS

A. Dataset Preparation

The use of transfer learning allows us to use a small dataset to improve the performance of the object detector. In this work, we fine tune YOLO's weights using a small dataset of 150 images of apples. The apple footage was collected from an apple orchard in sunny lighting. Half of the dataset are close up images of the apples, and the other half are images of apple trees. The dataset was manually annotated, with a couple of important considerations that improve the detector's performance.

Lighting and occlusion both pose challenges during detection and tracking. There are many cases where the trees' shadows dim the lighting of the apples, and the leaves or branches cover the apples. To tackle this problem, it is essential to include data where these cases occur, and annotate them appropriately. Table III illustrates different conditions and their respective annotations.

B. Weight Configurations

In well established computer vision neural networks, weights that have already been trained on large datasets are typically provided to accelerate implementation and the training process. First, we tested using YOLO's provided

weights, that have been pretrained on the MSCOCO dataset which consists of 80 classes. We only return detections that are classified as apples. However, the experiment led to a large amount of apples from other rows being detected, thus we developed and applied our correction mechanism, and discarded very small apples. Then, we configured the model settings to only contain the apple class and use the fine tuned weights. The model is trained for 5000 epochs with default training configurations. ResNet18's pretrained weights are not changed, as our goal with modifying the tracker is to make it a generalized solution.



Fig. 4: From the apple orchard video used, some apples from a back row tree are being detected and tracked

C. Results and Analysis

For evaluation, we compute the accuracy by comparing between the yield predicted by the model, and ground truth count which is performed manually by a human.

Table II shows the result of running our pipeline on the apple videos. We ran the experiments over 3 different model configurations: fine tuned weights, pretrained weights, and pretrained weights corrected by our mechanism. We compute the L1 Loss between the predicted and ground truth, and compute the accuracy using the following equation where GT is the ground truth, and L is the computed loss.

$$Accuracy = \frac{GT - L}{GT} * 100$$

As expected, using just YOLO's pretrained weights lead to a significant over count of 181 apples. This is due to how YOLO was trained on a massive dataset (MSCOCO) that featured apples of various sizes. Fig. 4 shows a video frame where a distant apple tree has 3 apples that were detected and tracked. At first, we aimed to solve the problem by increasing the minimum confidence needed for the detection to be tracked. However, that didn't help as there were other cases where the apple detection had lower confidence due to being occluded by other objects or, more commonly, blurred due to camera motion. Thus, the solution needed to tackle distant apples specifically. We ran the detection model on the same video and output all of the detections into the file, and we analyzed the detections against their respective apples to see the average size of a distant apple. Once we got an estimated size (30 pixels width and height) for the apple, we used that threshold in our correction mechanism.

Using YOLO's pretrained weights with the correction mechanism yielded a substantially better result, where the model under-counted by 43 apples, giving an accuracy of 87.43%. We looked at how the detection was behaving, and we observed that while distant apples were no longer being detected, the detection model would sometimes struggle with occluded apples. The detector itself also yielded average confidence levels, which did not cause an issue in our experiment due to our minimum confidence being fairly low, but would perform worse if the minimum confidence is increased. To address these issues, we use transfer learning and fine tune YOLO's weights using the dataset discussed previously. We make sure to include occluded apples and apples with different shading. We also changed the configuration of YOLO to only include the apple class instead of MSCOCO's 80 classes. Our fine tuned weights showed an improvement, with the model under-counting by only 29 apples, with our accuracy being 91.5%. Upon closer observation of this experiment, we found more occluded apples being detected appropriately, and the tracker was able to successfully identify and keep track of such detections. Figures 5, 6 and 7 illustrate the detection and tracking throughout three frames. More importantly, we learned that our correction mechanism was no longer needed, as we completely avoided annotating distant apples, which allowed us to remove the extra computation. However, there were still some apples that were heavily occluded that were not detected throughout all the video frames they were present in. Thus, apples that are almost completely hidden, despite any change in angle or lighting as the camera moves, pose a challenge to our pipeline.

V. CONCLUSION

We present a fully deep learning-based solution for fruit counting, and have proven its functionality with video data gathered from an apple orchard. Instead of performing counting on still images, we perform accurate counting on videos by designing a pipeline that consists of a detector that can recognize fruits and return their pixel-wise locations, and a tracker that associates between fruit detections across frames

and outputs a count of the fruits. We utilize YOLO for its accurate frame by frame detections with low processing times. We tested our apple data on pretrained weights, which performed detection correctly, but would detect apples in other rows leading to over counting. We solve this problem by acquiring the different possible sizes of apples through detection, and implementing a threshold mechanism that discards very small and distant apples accordingly. In a different configuration, we fine tune the weights of YOLO, ensuring the presence of occluded apples, and apples in different lighting for more versatile detection. The correction mechanism was not needed in that specific case as we do not annotate distant apples. We then integrate DeepSORT's tracking algorithm into our pipeline, which keeps track of objects based on their appearance features. The original implementation is limited to only person-reidentification, so we change the feature extractor to perform tracking on fruits. We implemented a pretrained ResNet18 model as the feature extractor for its robustness. We show that since ResNet has weights pretrained on ImageNet, which has 1000 classes, we did not need to modify the model or its weights for apples, nor would we need to for many other fruits. Our pipeline with fine tuned weights successfully counted the apples at 91.5% accuracy. For our future work, we plan to test the model on other fruits such as oranges, which would only require fine tuning the object detector. Moreover, since our primary challenge is during the detection stage, we also plan to use different detection techniques that can theoretically outperform YOLO, and test their practicality in fruit detection.

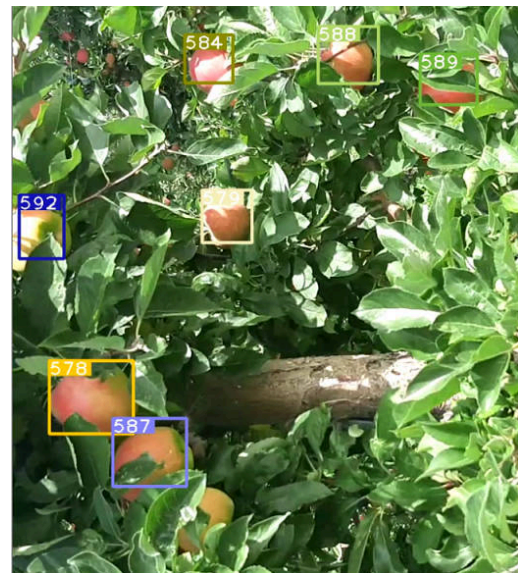


Fig. 5: Apples, despite not being completely visible, are detected and given track IDs

REFERENCES

- [1] D. Corwin and S. Lesch, "Application of soil electrical conductivity to precision agriculture: theory, principles, and guidelines," *Agronomy journal*, vol. 95, no. 3, pp. 455–471, 2003.

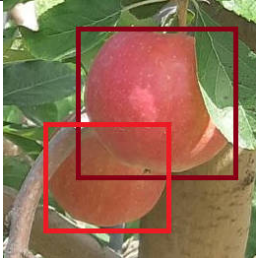
Case	Image	Description
1		Apple is partially occluded by a leaf; we include the leaf in the image so the model can learn apples with some leaf features
2		Half of the apple is occluded by a leaf; we include half the apple so the model can recognize what half an apple looks like and avoid over-training the model to start mistaking leaves for apples
3		Apple is heavily occluded by leaves, we only include the visible part of the apple, improving the models ability to recognize partial apple features
4		Apple is overlapping with another apple; though this problem is normally mended as the video progresses and the angle changes, we still want to ensure that the model is able to identify different apples detections that intersect with each other
5		Apples under different lighting: we address the variance in natural lighting where some spots are bright, and others are very dim by including apples under different lighting conditions

TABLE III: Different cases that affect the appearance of an apple that we include in our training dataset

- [2] L. Tian, J. F. Reid, and J. W. Hummel, "Development of a precision sprayer for site-specific weed management," *Transactions of the ASAE*, vol. 42, no. 4, p. 893, 1999.
- [3] N. Zhang, M. Wang, and N. Wang, "Precision agriculture—a worldwide overview," *Computers and electronics in agriculture*, vol. 36, no. 2-3, pp. 113–132, 2002.
- [4] V. Puri, A. Nayyar, and L. Raja, "Agriculture drones: A modern breakthrough in precision agriculture," *Journal of Statistics and Management Systems*, vol. 20, no. 4, pp. 507–518, 2017.
- [5] A. Syal, D. Garg, and S. Sharma, "A survey of computer vision methods for counting fruits and yield prediction," *International Journal of Computer Science Engineering*, vol. 2, no. 6, pp. 346–350, 2013.
- [6] A. Voulodimos, N. Doulamis, A. Doulamis, and E. Protopapadakis, "Deep learning for computer vision: A brief review," *Computational intelligence and neuroscience*, vol. 2018, 2018.
- [7] I. Sa, Z. Ge, F. Dayoub, B. Upcroft, T. Perez, and C. McCool, "Deepfruits: A fruit detection system using deep neural networks," *Sensors*, vol. 16, no. 8, p. 1222, 2016.
- [8] Y. Tian, G. Yang, Z. Wang, H. Wang, E. Li, and Z. Liang, "Apple detection during different growth stages in orchards using the improved yolo-v3 model," *Computers and electronics in agriculture*, vol. 157, pp. 417–426, 2019.
- [9] M. Rahnemoonfar and C. Sheppard, "Deep count: fruit counting based on deep simulated learning," *Sensors*, vol. 17, no. 4, p. 905, 2017.
- [10] Q. Wang, S. Nuske, M. Bergerman, and S. Singh, "Automated crop yield estimation for apple orchards," in *Experimental robotics*. Springer, 2013, pp. 745–758.
- [11] Z. S. Pothen and S. Nuske, "Texture-based fruit detection via images

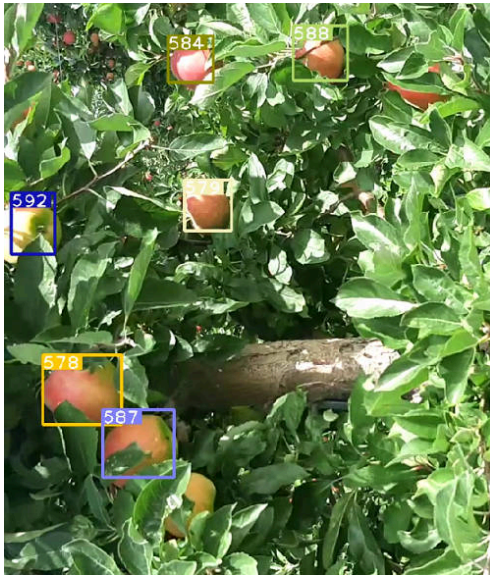


Fig. 6: The detected apples maintain their track IDs so long as they're detected. Apple ID 589 was not detected in this frame, thus is not shown

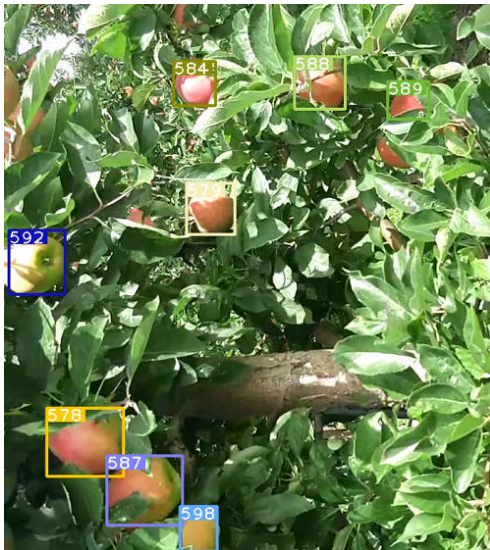


Fig. 7: The change in angle allowed the detector to detect a previously missed apple (ID 598), in addition, the tracker was able to re-identify apple ID 589 when it was detected again

using the smooth patterns on the fruit," in *2016 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2016, pp. 5171–5176.

- [12] P. Roy and V. Isler, "Surveying apple orchards with a monocular vision system," in *2016 IEEE international conference on automation science and engineering (CASE)*. IEEE, 2016, pp. 916–921.
- [13] S. Bargoti and J. P. Underwood, "Image segmentation for fruit detection and yield estimation in apple orchards," *Journal of Field Robotics*, vol. 34, no. 6, pp. 1039–1060, 2017.
- [14] C. Szegedy, S. Ioffe, V. Vanhoucke, and A. Alemi, "Inception-v4, inception-resnet and the impact of residual connections on learning," *arXiv preprint arXiv:1602.07261*, 2016.
- [15] S. W. Chen, S. S. Shivakumar, S. Dcunha, J. Das, E. Okon, C. Qu, C. J. Taylor, and V. Kumar, "Counting apples and oranges with deep learning:

A data-driven approach," *IEEE Robotics and Automation Letters*, vol. 2, no. 2, pp. 781–788, 2017.

- [16] J. Long, E. Shelhamer, and T. Darrell, "Fully convolutional networks for semantic segmentation," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2015, pp. 3431–3440.
- [17] X. Liu, S. W. Chen, S. Aditya, N. Sivakumar, S. Dcunha, C. Qu, C. J. Taylor, J. Das, and V. Kumar, "Robust fruit counting: Combining deep learning, tracking, and structure from motion," in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2018, pp. 1045–1052.
- [18] J. Redmon and A. Farhadi, "Yolov3: An incremental improvement," *arXiv preprint arXiv:1804.02767*, 2018.
- [19] X. Du, T.-Y. Lin, P. Jin, G. Ghiasi, M. Tan, Y. Cui, Q. V. Le, and X. Song, "Spinenet: Learning scale-permuted backbone for recognition and localization," 2019.
- [20] M. Tan and Q. V. Le, "Efficientnet: Rethinking model scaling for convolutional neural networks," 2019.
- [21] C.-Y. Wang, H.-Y. M. Liao, I.-H. Yeh, Y.-H. Wu, P.-Y. Chen, and J.-W. Hsieh, "Cspnet: A new backbone that can enhance learning capability of cnn," 2019.
- [22] P. Chao, C.-Y. Kao, Y.-S. Ruan, C.-H. Huang, and Y.-L. Lin, "Hardnet: A low memory traffic network," 2019.
- [23] A. Newell, K. Yang, and J. Deng, "Stacked hourglass networks for human pose estimation," 2016.
- [24] F. Yu, D. Wang, E. Shelhamer, and T. Darrell, "Deep layer aggregation," 2017.
- [25] M. Tan, R. Pang, and Q. V. Le, "Efficientdet: Scalable and efficient object detection," 2019.
- [26] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," 2017.
- [27] X. Zhou, D. Wang, and P. Krähenbühl, "Objects as points," 2019.
- [28] K. He, X. Zhang, S. Ren, and J. Sun, "Spatial pyramid pooling in deep convolutional networks for visual recognition," *Lecture Notes in Computer Science*, p. 346–361, 2014. [Online]. Available: http://dx.doi.org/10.1007/978-3-319-10578-9_23
- [29] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie, "Feature pyramid networks for object detection," 2016.
- [30] S. Liu, D. Huang, and Y. Wang, "Learning spatial fusion for single-shot object detection," 2019.
- [31] S. Woo, J. Park, J.-Y. Lee, and I. S. Kweon, "Cbam: Convolutional block attention module," 2018.
- [32] L.-C. Chen, G. Papandreou, I. Kokkinos, K. Murphy, and A. L. Yuille, "Deeplab: Semantic image segmentation with deep convolutional nets, atrous convolution, and fully connected crfs," 2016.
- [33] S. Liu, D. Huang, and Y. Wang, "Receptive field block net for accurate and fast object detection," 2017.
- [34] "Nvidia tensorrt," Aug 2020. [Online]. Available: <https://developer.nvidia.com/tensorrt>
- [35] J. Redmon, "Darknet: Open source neural networks in c," <http://pjreddie.com/darknet/>, 2013–2016.
- [36] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.
- [37] S. Ren, K. He, R. Girshick, and J. Sun, "Faster r-cnn: Towards real-time object detection with region proposal networks," in *Advances in neural information processing systems*, 2015, pp. 91–99.
- [38] B. Leibe, K. Schindler, and L. Van Gool, "Coupled detection and trajectory estimation for multi-object tracking," in *2007 IEEE 11th International Conference on Computer Vision*. IEEE, 2007, pp. 1–8.
- [39] N. Wojke, A. Bewley, and D. Paulus, "Simple online and realtime tracking with a deep association metric," in *2017 IEEE international conference on image processing (ICIP)*. IEEE, 2017, pp. 3645–3649.
- [40] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, "Simple online and realtime tracking," in *2016 IEEE International Conference on Image Processing (ICIP)*. IEEE, 2016, pp. 3464–3468.
- [41] X. Li, K. Wang, W. Wang, and Y. Li, "A multiple object tracking method using kalman filter," in *The 2010 IEEE international conference on information and automation*. IEEE, 2010, pp. 1862–1866.
- [42] L. Zheng, Z. Bie, Y. Sun, J. Wang, C. Su, S. Wang, and Q. Tian, "Mars: A video benchmark for large-scale person re-identification," in *European Conference on Computer Vision*. Springer, 2016, pp. 868–884.
- [43] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "Imagenet: A large-scale hierarchical image database," in *2009 IEEE conference on computer vision and pattern recognition*. Ieee, 2009, pp. 248–255.